

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería de Computación

Curso IC-1803 – Taller de Programación

Proyecto III – Árbol de Decisión Interactivo

Estudiantes

Amanda Torres Venegas y Génesis Villegas Umaña

Número de Carné

2026018764 y 2026018895

Fecha de entrega

Sábado 27 de junio, 2026

Profesor

Diego Andrés Mora Rojas

Grupo 01

Semestre 1- 2026

Índice

Descripción General del Sistema	3
Arquitectura del Proyecto	3
Representación del árbol dentro del programa	3
Clases	3
<i>Clase ArbolDecision (arbol.py)</i>	3
<i>Clase ManejadorArchivo (manejador_archivo.py)</i>	4
<i>Clase InterfazJuego (interfaz.py)</i>	5
Generalidades en la funcionalidad	5
<i>Relación entre clases</i>	5
<i>Recorrido del árbol durante una partida</i>	6
<i>Inserción al aprender</i>	6
<i>Manejo de archivos</i>	6
Árboles de prueba	6

Descripción General del Sistema

La aplicación implementa un árbol binario de decisión para el juego "Adivina en qué estoy pensando", con temática de comidas y frutas. El sistema realiza preguntas de Sí/No para adivinar un elemento pensado por el usuario, y aprende cuando falla, guardando el nuevo conocimiento en un archivo JSON.

El proyecto aplica separación de responsabilidades: la lógica del árbol, la interfaz gráfica y el manejo de archivos están en módulos independientes.

Además, para el desarrollo de este proyecto se utilizó Claude (Anthropic) como asistente de inteligencia artificial, entre ello para la revisión de código, recomendaciones de diseño y generación de los archivos JSON de prueba.

Arquitectura del Proyecto

El proyecto consta de dos archivos Python:

Archivo	Contenido
árbol.py	Clase ArbolDecision — lógica del árbol, recorrido y aprendizaje
principal.py	Clase ManejadorArchivo — guardar y cargar archivos JSON
interfaz.py	Clase InterfazJuego — pantallas, botones y flujo visual
manejador_archivo.py	Punto de entrada — solo arranca la ventana

Representación del árbol dentro del programa

El árbol se representa con listas anidadas, siguiendo la misma estructura vista en clase:

[raiz, hijo_izquierdo, hijo_derecho]

Los nodos internos son listas de 3 elementos (los nodos hoja (respuestas finales) son strings directamente). Ejemplo:

["¿Es una fruta?", ["¿Es cítrica?", "naranja", "mango"], "hamburguesa"]

Clases

Clase ArbolDecision (arbol.py)

Contiene toda la lógica del juego (modela, gestiona y actualiza dinámicamente el árbol de decisión):

Atributos:

- arbol — lista anidada que representa el árbol completo.
- nodo_actual — referencia al nodo donde se encuentra la partida activa.

Métodos base:

- `atomo(x)` — retorna True si x es hoja (string sin hijos).
- `raiz(arbol)` — retorna el contenido del nodo raíz.
- `hijoizq(arbol)` — retorna la rama Sí.
- `hijoder(arbol)` — retorna la rama No.
- `hoja(nodo)` — retorna True si el nodo no tiene hijos.
- `construir (centro, hi, hd)` — construye un nodo nuevo.

Métodos de partida:

- `reiniciar_partida()` — vuelve `nodo_actual` a la raíz.
- `obtener_pregunta_actual()` — retorna el texto del nodo actual.
- `es_hoja_actual()` — retorna True si el nodo actual es hoja.
- `responder(respuesta_si)` — avanza por la rama Sí o No según la respuesta.

Métodos de aprendizaje:

- `aprender(respuesta_correcta, nueva_pregunta, nueva_es_si)` — reemplaza la hoja incorrecta por una nueva pregunta con dos hijos.
- `_reemplazar(arbol, objetivo, reemplazo)` — recorre el árbol recursivamente hasta encontrar el nodo objetivo y lo reemplaza.

Métodos de serialización:

- `a_diccionario()` — retorna el árbol como lista para guardarlo en JSON.
- `desde_diccionario(datos)` — reconstruye el árbol desde los datos cargados.

Clase ManejadorArchivo (manejador_archivo.py)

Gestiona la persistencia del árbol. No contiene lógica del árbol ni de la interfaz. Además, no tiene atributos, toda su lógica opera sobre los parámetros que recibe.

Métodos:

- `guardar(arbol, ruta)` — serializa el árbol a JSON y lo escribe en disco. Retorna (True, "") si fue exitoso o (False, mensaje) si falló.
- `cargar(arbol, ruta)` — lee el JSON y reconstruye el árbol. Retorna (True, "") o (False, mensaje).
- `_validar_estructura(datos)` — valida recursivamente que el JSON tenga el formato correcto. Un nodo válido es un string no vacío o una lista de exactamente 3 elementos.

Manejo de errores: captura archivo inexistente, archivo vacío, JSON malformado, estructura incorrecta y errores de permisos.

Clase InterfazJuego (interfaz.py)

Gestiona las 5 pantallas del juego usando Tkinter. Delega toda la lógica a ArbolDecision y ManejadorArchivo.

Atributos:

- root: referencia a la ventana principal de Tkinter.
- Árbol: instancia de ArbolDecision con el árbol en juego.
- Manejador: instancia de ManejadorArchivo para guardar y cargar.
- archivo_actual: ruta del archivo JSON cargado, o None si se usa el árbol predeterminado.

Pantallas:

- Pantalla 1 — Inicio: título, instrucciones, botones para jugar, cargar archivo, guardar árbol actual y salir.
- Pantalla 2 — Juego: muestra la pregunta actual y botones Sí/No.
- Pantalla 3 — Victoria: mensaje cuando el sistema adivina correctamente.
- Pantalla 4 — Aprendizaje: formulario con 3 campos validados para enseñar al sistema.
- Pantalla 5 — Confirmación: mensaje de que el árbol fue actualizado y guardado.

Métodos principales:

- mostrar_pantalla_inicio() — renderiza la pantalla de bienvenida.
- iniciar_partida() — reinicia el árbol y muestra la primera pregunta.
- mostrar_pregunta() — renderiza el nodo actual.
- procesar_respuesta(respuesta_si) — avanza en el árbol o va a victoria/aprendizaje.
- mostrar_formulario_aprendizaje() — formulario con validaciones.
- guardar_arbol_automatico() — guarda sin pedir ruta si ya hay archivo cargado.

Generalidades en la funcionalidad:

Relación entre clases:

- InterfazJuego usa ArbolDecision para leer preguntas, avanzar y aprender.
- InterfazJuego usa ManejadorArchivo para cargar y guardar desde la GUI.
- ManejadorArchivo usa ArbolDecision para llamar a_diccionario() y desde_diccionario().
- ArbolDecision no conoce ni a InterfazJuego ni a ManejadorArchivo.

Recorrido del árbol durante una partida

1. Se parte de la raíz. nodo_actual apunta al nodo en juego.
2. Si nodo_actual no es hoja, se muestra su contenido como pregunta.
3. Según la respuesta, nodo_actual avanza a hijoizq (Sí) o hijoder (No).
4. Cuando nodo_actual es hoja, se muestra su contenido como intento de respuesta.

Inserción al aprender

Cuando el sistema falla en una hoja:

1. Se guarda el contenido de la hoja como respuesta incorrecta.
2. Se construye un nuevo nodo lista de 3 elementos: la nueva pregunta, y los dos hijos según si la respuesta correcta va en la rama Sí o No.
3. El método _reemplazar recorre el árbol recursivamente hasta encontrar la hoja incorrecta y la sustituye por el nuevo nodo.

Manejo de archivos

Cómo se guarda el árbol: Al terminar el aprendizaje, guardar_automatico() llama a ManejadorArchivo.guardar(), que convierte el árbol a JSON con json.dump() y lo escribe en disco. Si el usuario ya tenía un archivo cargado, se sobrescribe ese mismo archivo. Si jugó con el árbol predeterminado sin cargar archivo, se abre un diálogo para que elija dónde guardar. También existe un botón de guardado manual en el menú de inicio.

Cómo se carga desde archivo: Desde el menú de inicio, el usuario selecciona un archivo JSON. ManejadorArchivo.cargar() verifica que el archivo exista, no esté vacío y sea JSON válido. Luego _validar_estructura() comprueba recursivamente que cada nodo sea un string no vacío o una lista de exactamente 3 elementos. Si todo es correcto, desde_diccionario() reemplaza el árbol en memoria y reinicia nodo_actual a la raíz. Si hay algún error, se muestra un mensaje y se continúa con el árbol predeterminado.

Árboles de prueba

Para el proyecto se implementaron los siguientes casos basados en la temática de comida:

Archivo	Tema	Respuestas
frutas_tropicales.json	Frutas según clima y tipo	15
platos_latinoamericanos.json	Platos típicos de Latinoamérica	14
postres_dulces.json	Postres por método de preparación	15
street_food.json	Comida callejera popular	12
snacks_y_botanitas.json	Snacks salados y dulces	27